In [1]:
```python
import pandas as pd
import numpy as np
```

In [2]:
```python
df=pd.read_csv(r"D:\Downloads\loan_approval_dataset.csv")
```

In [5]:
```python
df
```

Out[5]:

	loan_id	no_of_dependents	education	self_employed	income_annum	loan_amount	loan_term	cibil_score	residential_assets_value	commercial_assets_value	luxury_assets_value	bank_asset_value	loan_stat
0	1	2	Graduate	No	9600000	29900000	12	778	2400000	17600000	22700000	8000000	Approv
1	2	0	Not Graduate	Yes	4100000	12200000	8	417	2700000	2200000	8800000	3300000	Reject
2	3	3	Graduate	No	9100000	29700000	20	506	7100000	4500000	33300000	12800000	Reject
3	4	3	Graduate	No	8200000	30700000	8	467	18200000	3300000	23300000	7900000	Reject
4	5	5	Not Graduate	Yes	9800000	24200000	20	382	12400000	8200000	29400000	5000000	Reject
...	...	...	...	...	...	...	...	...	...	...	...	...	
4264	4265	5	Graduate	Yes	1000000	2300000	12	317	2800000	500000	3300000	800000	Reject
4265	4266	0	Not Graduate	Yes	3300000	11300000	20	559	4200000	2900000	11000000	1900000	Approv
4266	4267	2	Not Graduate	No	6500000	23900000	18	457	1200000	12400000	18100000	7300000	Reject
4267	4268	1	Not Graduate	No	4100000	12800000	8	780	8200000	700000	14100000	5800000	Approv
4268	4269	1	Graduate	No	9200000	29700000	10	607	17800000	11800000	35700000	12000000	Approv

4269 rows × 13 columns

In [7]:
```python
df.head()
```

Out[7]:

	loan_id	no_of_dependents	education	self_employed	income_annum	loan_amount	loan_term	cibil_score	residential_assets_value	commercial_assets_value	luxury_assets_value	bank_asset_value	loan_status
0	1	2	Graduate	No	9600000	29900000	12	778	2400000	17600000	22700000	8000000	Approved
1	2	0	Not Graduate	Yes	4100000	12200000	8	417	2700000	2200000	8800000	3300000	Rejected
2	3	3	Graduate	No	9100000	29700000	20	506	7100000	4500000	33300000	12800000	Rejected
3	4	3	Graduate	No	8200000	30700000	8	467	18200000	3300000	23300000	7900000	Rejected
4	5	5	Not Graduate	Yes	9800000	24200000	20	382	12400000	8200000	29400000	5000000	Rejected

In [9]:
```python
df.shape
```

Out[9]:
```
(4269, 13)
```

In [11]:
```python
df.columns
```

Out[11]:
```
Index(['loan_id', ' no_of_dependents', ' education', ' self_employed',
       ' income_annum', ' loan_amount', ' loan_term', ' cibil_score',
       ' residential_assets_value', ' commercial_assets_value',
       ' luxury_assets_value', ' bank_asset_value', ' loan_status'],
      dtype='object')
```

In [13]:
```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4269 entries, 0 to 4268
Data columns (total 13 columns):
 #   Column                     Non-Null Count  Dtype 
---  ------                     --------------  ----- 
 0   loan_id                    4269 non-null   int64 
 1    no_of_dependents          4269 non-null   int64 
 2    education                 4269 non-null   object
 3    self_employed             4269 non-null   object
 4    income_annum              4269 non-null   int64 
 5    loan_amount               4269 non-null   int64 
 6    loan_term                 4269 non-null   int64 
 7    cibil_score               4269 non-null   int64 
 8    residential_assets_value  4269 non-null   int64 
 9    commercial_assets_value   4269 non-null   int64 
 10   luxury_assets_value       4269 non-null   int64 
 11   bank_asset_value          4269 non-null   int64 
 12   loan_status               4269 non-null   object
dtypes: int64(10), object(3)
memory usage: 433.7+ KB
```

In [15]:
```python
df.describe()
```

Out[15]:

	loan_id	no_of_dependents	income_annum	loan_amount	loan_term	cibil_score	residential_assets_value	commercial_assets_value	luxury_assets_value	bank_asset_value
count	4269.000000	4269.000000	4.269000e+03	4.269000e+03	4269.000000	4269.000000	4.269000e+03	4.269000e+03	4.269000e+03	4.269000e+03
mean	2135.000000	2.498712	5.059124e+06	1.513345e+07	10.900445	599.936051	7.472617e+06	4.973155e+06	1.512631e+07	4.976692e+06
std	1232.498479	1.695910	2.806840e+06	9.043363e+06	5.709187	172.430401	6.503637e+06	4.388966e+06	9.103754e+06	3.250185e+06
min	1.000000	0.000000	2.000000e+05	3.000000e+05	2.000000	300.000000	-1.000000e+05	0.000000e+00	3.000000e+05	0.000000e+00
25%	1068.000000	1.000000	2.700000e+06	7.700000e+06	6.000000	453.000000	2.200000e+06	1.300000e+06	7.500000e+06	2.300000e+06
50%	2135.000000	3.000000	5.100000e+06	1.450000e+07	10.000000	600.000000	5.600000e+06	3.700000e+06	1.460000e+07	4.600000e+06
75%	3202.000000	4.000000	7.500000e+06	2.150000e+07	16.000000	748.000000	1.130000e+07	7.600000e+06	2.170000e+07	7.100000e+06
max	4269.000000	5.000000	9.900000e+06	3.950000e+07	20.000000	900.000000	2.910000e+07	1.940000e+07	3.920000e+07	1.470000e+07

In [17]:
```python
df.isnull().sum()
```

Out[17]:
```
loan_id                      0
 no_of_dependents            0
 education                   0
 self_employed               0
 income_annum                0
 loan_amount                 0
 loan_term                   0
 cibil_score                 0
 residential_assets_value    0
 commercial_assets_value     0
 luxury_assets_value         0
 bank_asset_value            0
 loan_status                 0
dtype: int64
```

In [19]:
```python
df.duplicated().sum()
```

Out[19]:
```
0
```

In [21]:
```python
df.columns=df.columns.str.replace(" ","")
```

In [23]:
```python
df['loan_status'].unique()
```

Out[23]:
```
array([' Approved', ' Rejected'], dtype=object)
```

In [25]:
```python
df['loan_status']=df['loan_status'].astype(str).str.strip()
```

In [27]:
```python
df['loan_status'].unique()
```

Out[27]:
```
array(['Approved', 'Rejected'], dtype=object)
```

In [29]:
```python
df['education'].unique()
```

Out[29]:
```
array([' Graduate', ' Not Graduate'], dtype=object)
```

In [31]:
```python
df['education']=df['education'].astype(str).str.strip()
```

In [33]:
```python
df['loan_status']=df['loan_status'].map({
    'Approved':1,
    'Rejected':0
})
```

In [35]:
```python
df['loan_status'].value_counts()
```

Out[35]:
```
loan_status
1    2656
0    1613
Name: count, dtype: int64
```

Feature selection

In [38]:
```python
Y=df[['loan_status']]
X=df.drop(labels=['loan_status','loan_id'],axis=1)
```

In [40]:
```python
Y
```

Out[40]:

	loan_status
0	1
1	0
2	0
3	0
4	0
...	...
4264	0
4265	1
4266	0
4267	1
4268	1

4269 rows × 1 columns

In [42]:
```python
X
```

Out[42]:

	no_of_dependents	education	self_employed	income_annum	loan_amount	loan_term	cibil_score	residential_assets_value	commercial_assets_value	luxury_assets_value	bank_asset_value
0	2	Graduate	No	9600000	29900000	12	778	2400000	17600000	22700000	8000000
1	0	Not Graduate	Yes	4100000	12200000	8	417	2700000	2200000	8800000	3300000
2	3	Graduate	No	9100000	29700000	20	506	7100000	4500000	33300000	12800000
3	3	Graduate	No	8200000	30700000	8	467	18200000	3300000	23300000	7900000
4	5	Not Graduate	Yes	9800000	24200000	20	382	12400000	8200000	29400000	5000000
...	...	...	...	...	...	...	...	...	...	...	...
4264	5	Graduate	Yes	1000000	2300000	12	317	2800000	500000	3300000	800000
4265	0	Not Graduate	Yes	3300000	11300000	20	559	4200000	2900000	11000000	1900000
4266	2	Not Graduate	No	6500000	23900000	18	457	1200000	12400000	18100000	7300000
4267	1	Not Graduate	No	4100000	12800000	8	780	8200000	700000	14100000	5800000
4268	1	Graduate	No	9200000	29700000	10	607	17800000	11800000	35700000	12000000

4269 rows × 11 columns

In [44]:
```python
categorical_features=[]
numeric_features=[]
for i in X.columns:
    if df[i].dtypes=='object':
        categorical_features.append(i)
    else:
        numeric_features.append(i)
```

In [46]:
```python
categorical_features
```

Out[46]:
```
['education', 'self_employed']
```

In [48]:
```python
numeric_features
```

Out[48]:
```
['no_of_dependents',
 'income_annum',
 'loan_amount',
 'loan_term',
 'cibil_score',
 'residential_assets_value',
 'commercial_assets_value',
 'luxury_assets_value',
 'bank_asset_value']
```

Train test split

In [51]:
```python
from sklearn.model_selection import train_test_split
xtrain,xtest,ytrain,ytest=train_test_split(X,Y,test_size=0.2,random_state=42)
```

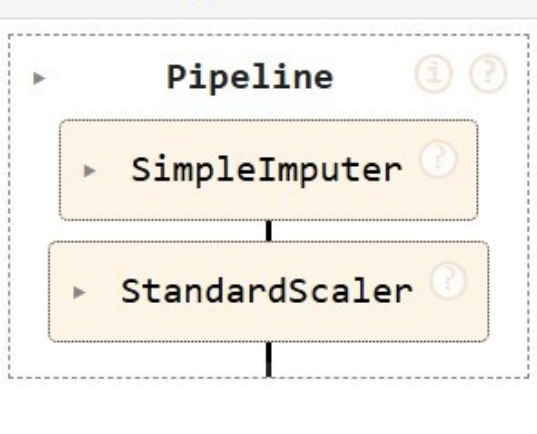
Preprocessing

In [53]:
```python
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder,StandardScaler


numeric_pipeline=Pipeline([
    ('imputer',SimpleImputer(strategy='mean')),
    ('scaler',StandardScaler())
])
```

In [54]:
```python
numeric_pipeline
```

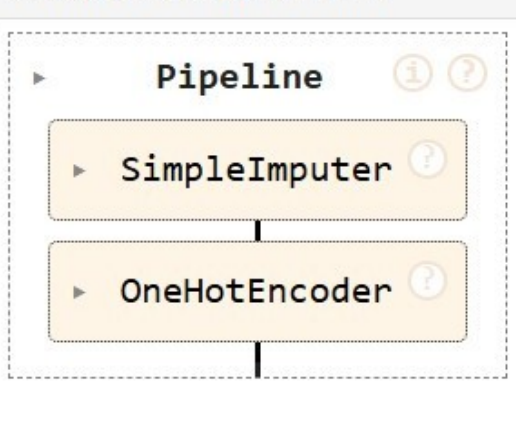
Out[54]:



In [55]:
```python
categorical_pipeline=Pipeline([
    ('imputer',SimpleImputer(strategy='most_frequent')),
    ('encoder',OneHotEncoder(handle_unknown='ignore'))
])
```

In [60]:
```python
categorical_pipeline
```

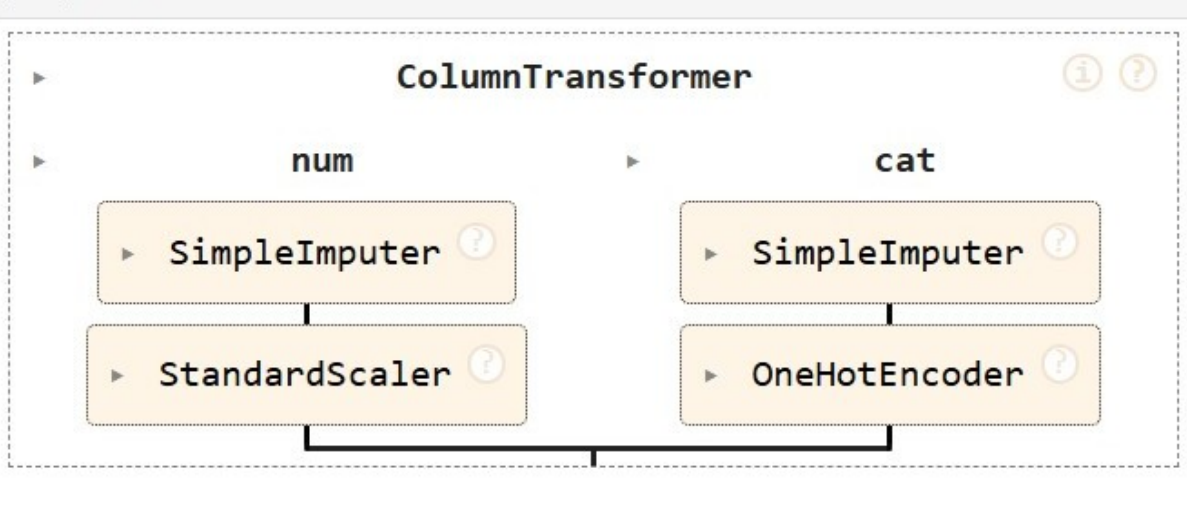
Out[60]:



In [62]:
```python
preprocessor=ColumnTransformer([
    ('num',numeric_pipeline,numeric_features),
    ('cat',categorical_pipeline,categorical_features)
])
```

In [64]:
```python
preprocessor
```

Out[64]:

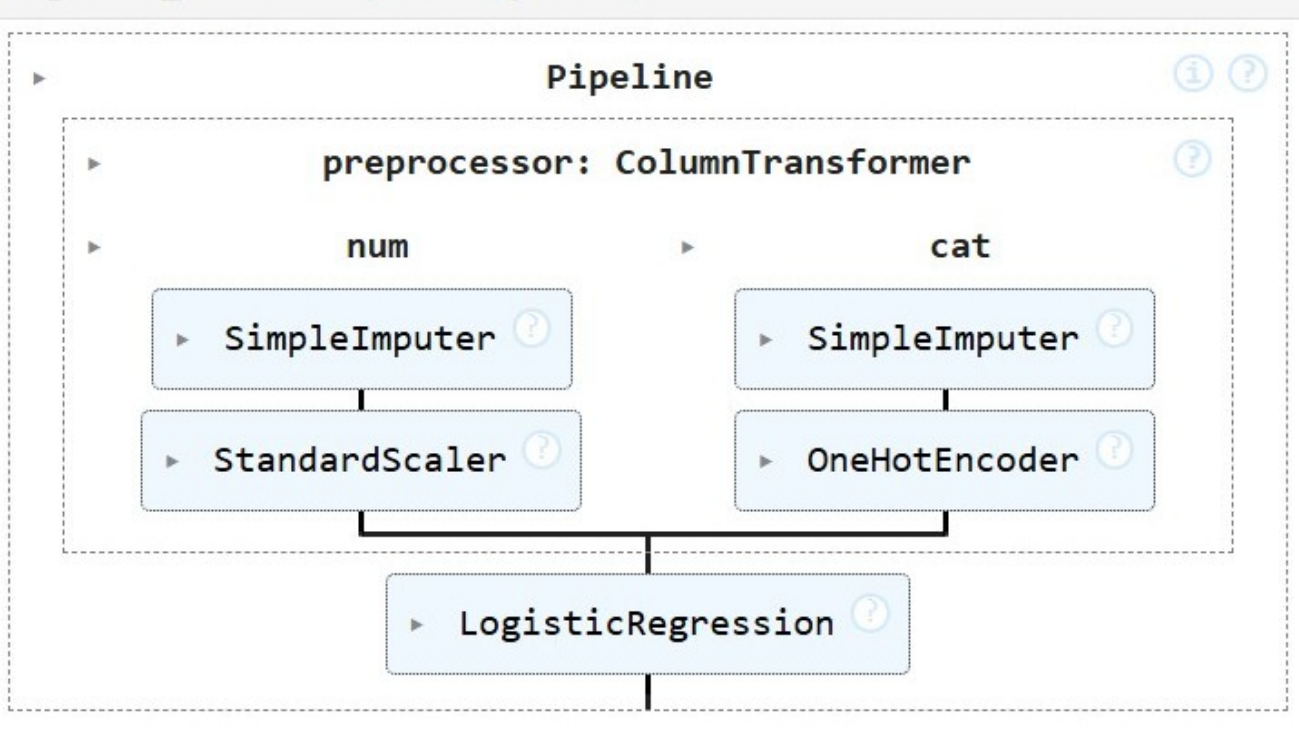


In [66]:
```python
from sklearn.linear_model import LogisticRegression

logistic_model=Pipeline([
    ('preprocessor',preprocessor),
    ('model',LogisticRegression(max_iter=1000))
])
```

In [72]:
```python
logistic_model.fit(xtrain,ytrain)
```

Out[72]:



In [74]:
```python
from warnings import filterwarnings
filterwarnings('ignore')
```

In [76]:
```python
y_pred_lr=logistic_model.predict(xtest)
```

In [78]:
```python
y_pred_lr
```

Out[78]:
```
array([0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0,
       0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1,
       0, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1,
       0, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1,
       1, 1, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1, 1, 0,
       1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0,
       1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1,
       1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1, 0, 1,
       1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0,
       1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0,
       1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1,
       1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1,
       1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0,
       1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1,
       0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0,
       0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0,
       1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0, 1, 1,
       0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1,
       0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1, 1, 0, 1,
       1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1,
       0, 1, 1, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0,
       1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1,
       0, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 0, 1,
       1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1,
       1, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1,
       0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 1, 1,
       0, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1,
       0, 1, 1, 0, 0, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0,
       1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 1, 0,
       0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0,
       0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1,
       0, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1,
       0, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 0, 1,
       0, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 1, 1, 0,
       1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1], dtype=int64)
```

Evaluate

In [81]:
```python
from sklearn.metrics import accuracy_score,classification_report,confusion_matrix

accuracy_score(ytest,y_pred_lr)
```

Out[81]:
```
0.905152224824356
```

In [83]:
```python
classification_report(ytest,y_pred_lr)
```

Out[83]:
```
'              precision    recall  f1-score   support\n\n           0       0.88      0.86      0.87       318\n           1       0.92      0.93      0.92       536\n\n    accuracy                           
0.91       854\n   macro avg       0.90      0.90      0.90       854\nweighted avg       0.90      0.91      0.90       854\n'
```

In [85]:
```python
import seaborn as sb
import matplotlib.pyplot as plt
cm=confusion_matrix(ytest,y_pred_lr)
sb.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)
plt.xlabel("Predicted")
plt.ylabel('Actual')
plt.title('Logistic regression Confusion Matrix')
plt.show()
```